



✓ [Tham Lam]. Bài 16. Egyptian fractions 🔒 DSA 📄 View as PDF

Một phân số đơn vị nếu tử số của phân số đó là 1. Mọi phân số nguyên dương đều có thể biểu diễn thành tổng các phân số đơn vị. Ví dụ $2/3 = 1/2 + 1/6$. Cho phân số nguyên dương P/Q , hãy biểu diễn phân số nguyên dương thành tổng phân số đơn vị với số hạng tử là ít nhất.

[Submit solution](#)[My submissions](#)[All submissions](#)[Best submissions](#)

Đầu vào

1 dòng duy nhất chứa 2 số P, Q

Giới hạn

$1 \leq P, Q \leq 400$

Đầu ra

Đưa ra đáp án trên 1 dòng

Ví dụ :

Input 01

5 6

[Copy](#)

Output 01

$1/2 + 1/3$

[Copy](#)

✓ **Points:** 1.00 (partial)

🕒 **Time limit:** 1.0s

📄 **Memory limit:** 256M

⌨ **Input:** stdin

🖨 **Output:** stdout

📝 **Problem source:**
28Tech

➤ **Problem type**

▼ **Allowed languages**
C, C++, Java, Kotlin,
Pascal, PyPy, Python,
Scratch

💬 Comments

[Report an issue](#)



commented 8:13:10 pm, 11/09/2025

dungdeptrai

Full AC...Không biết có phải tham lam không nhưng mãi mới nhìn ra hướng giải...

```
include <bits/stdc++.h>  
define fastIO  
iosbase::syncwith_stdio(false),  
cin.tie(nullptr);  
define MOD 1000000007
```

```
using namespace std;
```

```
int gcd(int a, int b) { if (b == 0) { return a; } return gcd(b, a % b); }
```

```
struct fraction { long long tu, mau;
```

[Copy](#)



```
{  
}  
  
fraction(long long t, long long m)  
{  
    tu = t;  
    mau = m;  
    rutGon();  
}  
  
string toString()  
{  
    stringstream builder;  
    builder << tu << '/' << mau;  
    return builder.str();  
}  
  
void rutGon()  
{  
    int tmp = gcd(tu, mau);  
    mau /= tmp;  
    tu /= tmp;  
}  
  
friend fraction operator-(const fraction &x, const  
fraction &y)  
{  
    fraction res;  
    res.mau = x.mau * y.mau;  
    res.tu = x.tu * y.mau - y.tu * x.mau;  
    res.rutGon();  
    return res;  
}
```

```
};
```

```
int main() { fastIO;
```

```
ifndef ONLINE_JUDGE
```

[Copy](#)



```
freopen("output.txt", "w", stdout);
```

endif

```
int p, q;
cin >> p >> q;
fraction a{p, q};
vector<fraction> listFraction;

while (a.tu != 1)
{
    long long x = (a.mau + a.tu - 1) / a.tu;
    fraction tmp{1, x};
    listFraction.push_back(tmp);
    a = a - tmp;
}

listFraction.push_back(a);

sort(listFraction.begin(), listFraction.end(), []
(const fraction &x, const fraction &y)
    { return x.mau < y.mau; });

for (int i = 0; i < (int)listFraction.size(); i++)
{
    cout << listFraction[i].toString();
    if (i < (int)listFraction.size() - 1)
    {
        cout << " + ";
    }
}

return 0;

}
```

[Copy](#)



Hello, Nguyễn Bảo Toàn.

Post!

Được phát triển bởi **28TECH.COM.VN**